

May 22th , 2026



**Database system
SWEG2108**

College of Electrical and Mechanical Engineering
Department of Software Engineering

FOOD DELIVERY DATABASE SYSTEM: PROJECT REPORT

GROUP MEMBERS

ID

1. AFOMIYA ASCHENAKI	ETS 0111/17
2. AKSAH ATLAU	ETS 0121/17
3. ARSEMA YESHIDNBER	ETS 0199/17
4. BEREKET DERESSE	ETS 0262/17
5. BETELHEM NURLACHEW	ETS 0284/17
6. BETELHEM SOLOMON	ETS 0286/17
7. MADING MAJOK	ETS 1876/17

QUICKBITE – INTELLIGENT FOOD DELIVERY SYSTEM

Submitted To: Dr. Yaynshet Medhin
Addis Ababa, Ethiopia

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

In the modern digital era, the rapid advancement of Information and Communication Technologies (ICT) has significantly transformed the way businesses operate, especially in the food service industry. Food delivery systems have evolved from simple telephone-based ordering methods into highly integrated, data-driven platforms. These platforms connect customers, restaurants, delivery personnel, and system administrators within a single digital environment, enabling faster and more efficient service delivery.

This transformation is largely driven by several real-world factors such as increasing urbanization, busy lifestyles, and the growing demand for convenience. Today, customers expect to order food anytime and from anywhere, making digital food delivery systems an essential part of everyday life. Behind these systems lies a large amount of data generated from various activities such as browsing menus, placing orders, making payments, and tracking deliveries.

The effectiveness of a food delivery system depends heavily on how well this data is managed. A poorly designed database can lead to delays, incorrect orders, and loss of important information. On the other hand, a well-structured database ensures smooth operations, accurate transactions, and better user experience.

Traditionally, relational databases such as MySQL have been used to manage structured data because of their reliability and strong consistency features (ACID properties). However, modern applications also require flexibility and scalability to handle large volumes of diverse data. This is where NoSQL databases like MongoDB become useful, as they allow storage of unstructured and semi-structured data in a flexible format.

This project focuses on designing and implementing a hybrid database system for a food delivery service using both MySQL and MongoDB. The aim is to combine the strengths of both approaches to build a system that is efficient, scalable, and suitable for real-world applications.

1.2 Problem Statement

Despite the rapid growth of food delivery platforms, many existing systems still face serious challenges related to data management and system performance. These issues are often caused by poor database design, lack of integration between system components, and inability to handle increasing amounts of data.

One major problem is data inconsistency, where the same information is stored in multiple places but not updated properly. For example, a customer's address may be different in two tables, or menu prices may not be synchronized across the system. This creates confusion and affects the overall reliability of the system.

Another issue is scalability. As the number of users and transactions increases, systems that rely only on rigid relational structures may struggle to maintain performance. In addition, inefficient queries can slow down order processing, leading to delays and poor customer experience.

The key problems addressed in this project include:

- Data fragmentation and inconsistency between different system components
- Data redundancy due to poor database design
- Difficulty in managing complex relationships between entities
- Scalability limitations when handling large datasets
- Slow data retrieval affecting real-time operations
- Lack of effective reporting and analysis capabilities

This project aims to solve these challenges by designing a properly normalized relational database and integrating it with a flexible NoSQL database.

1.3 Project Motivation

The motivation behind this project goes beyond simply fulfilling academic requirements. It is based on understanding how important database systems are in real-life applications.

In a food delivery system, even a small error in the database can lead to serious consequences. For customers, it may result in receiving the wrong order or experiencing delays. For restaurants, it may lead to lost revenue and reduced customer trust. For delivery personnel, it may cause confusion and inefficiency in operations.

This project is therefore motivated by the need to design a system that is not only technically correct but also reliable and practical. It aims to bridge the gap between theoretical knowledge learned in class and its real-world application, ensuring that the system can support actual business needs.

1.4 Objectives of the Study

General Objective

The general objective of this project is to design and implement an efficient and scalable food delivery database system using both relational (MySQL) and non-relational (MongoDB) technologies.

Specific Objectives

To achieve the general objective, the project focuses on the following:

- To analyze the data requirements of a food delivery system
- To identify entities, attributes, and relationships involved in the system
- To design an Entity-Relationship (ER) diagram representing the system structure
- To develop a relational schema and normalize it up to Boyce-Codd Normal Form (BCNF)
- To implement the database using MySQL for structured data
- To implement MongoDB for flexible and scalable data storage
- To insert sample data and perform queries
- To generate reports for analysis and decision-making
- To evaluate system performance and efficiency

1.5 Scope and Delimitations

The scope of this project focuses mainly on the database design and implementation of a food delivery system. It includes the core functionalities required to manage the system effectively.

The system covers:

- Customer registration and profile management
- Restaurant and menu management
- Order placement and tracking
- Delivery personnel assignment
- Payment recording and management

However, the project has certain limitations, including:

- No development of a full web or mobile user interface

- No real-time GPS tracking (only simulated)
- No integration with real online payment systems
- Limited advanced analytics and recommendation features

These limitations are set to keep the focus on database design and ensure a clear understanding of data structures and relationships.

1.6 Significance of the Study

This project is important both academically and practically.

- ✓ From an academic perspective, it helps in applying database concepts such as ER modeling, normalization, and system design to a real-world problem. It also introduces the concept of using multiple database systems together.
- ✓ From a practical perspective, the system provides a foundation for building scalable and efficient food delivery applications. It demonstrates how proper database design can improve system performance and user experience.

The key benefits include:

- Better organization and management of data
- Reduction of data redundancy and inconsistency
- Improved efficiency in order processing
- Enhanced decision-making through reports
- Scalability to handle growing data
- Flexibility in managing different types of data

1.7 Methodology

This project follows a structured approach based on the Database Development Life Cycle (DDLC). Each stage is carefully executed to ensure a reliable system.

The main steps include:

- **Requirement Analysis:** Identifying system needs and data requirements
- **Conceptual Design:** Creating ER diagrams to represent entities and relationships
- **Logical Design:** Developing relational schema and applying normalization
- **Implementation:** Building the database using MySQL and MongoDB
- **Testing and Evaluation:** Checking system performance and correctness

This methodology ensures that the system is well-organized and meets its objectives.

1.8 Tools and Technologies Used

The following tools and technologies are used in this project:

- **MySQL:** For relational database implementation and structured data management
- **MongoDB:** For NoSQL database and flexible data storage
- **Draw.io:** For designing ER diagrams
- **GitHub:** For version control and collaboration
- **Google Docs:** For documentation and report preparation

CHAPTER 2: DATABASE DESIGN

This chapter presents the complete database design of the proposed hybrid food delivery management system. The design phase focuses on transforming the identified business requirements into a structured database model capable of supporting efficient data storage, retrieval, processing, and analysis. The chapter includes the Entity Relationship (ER) Diagram, Logical Schema Design, entity descriptions, attribute explanations, relationship analysis, normalization process, and input/output design considerations.

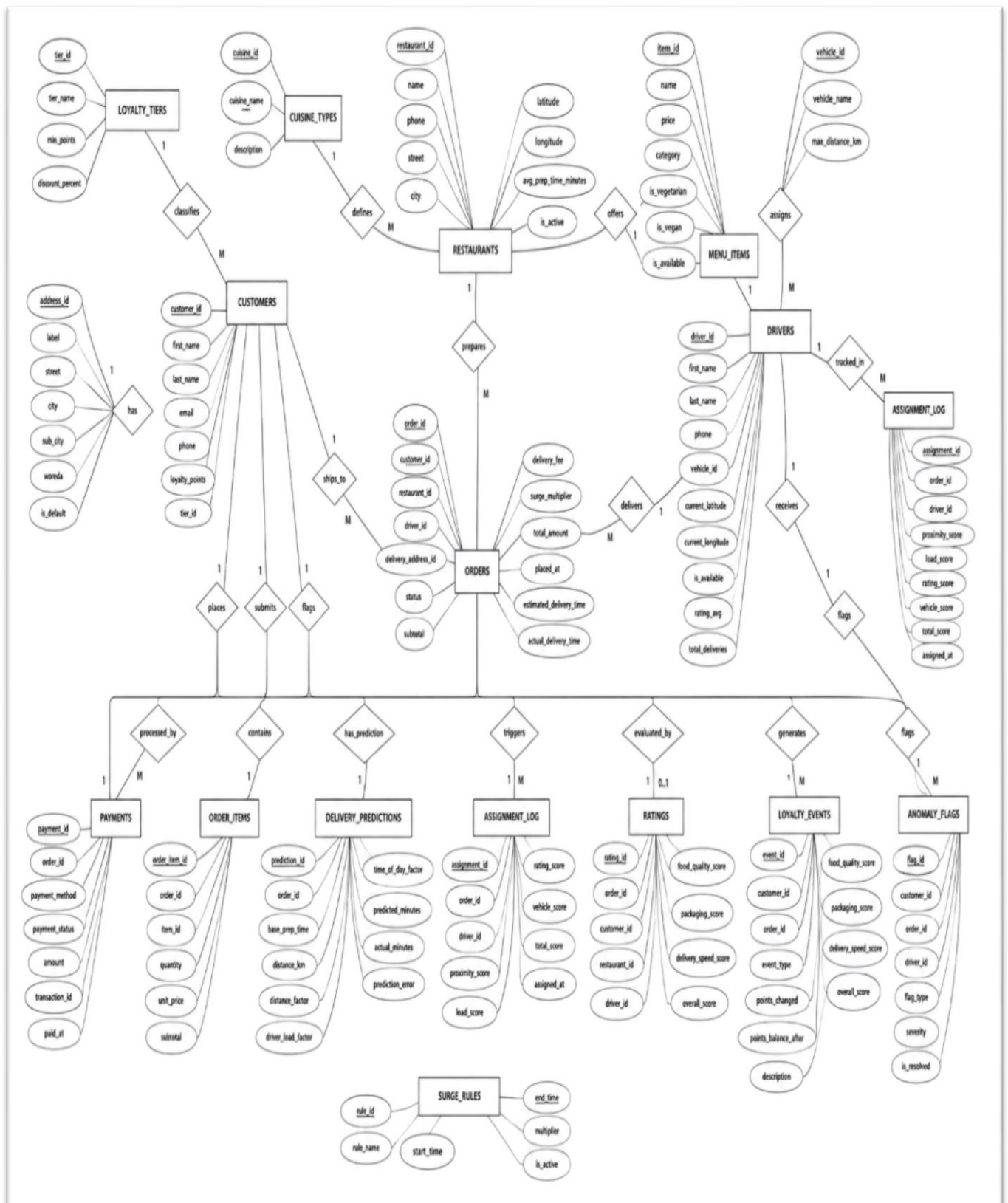
The database was designed using relational database principles to ensure data consistency, integrity, scalability, and reduced redundancy. The system integrates operational data such as customers, restaurants, orders, drivers, menu items, payments, loyalty programs, delivery predictions, and anomaly detection mechanisms into a unified structure.

2.1 Entity Relationship (ER) Diagram Description

The Entity Relationship Diagram (ERD) provides a conceptual representation of the food delivery database system. It illustrates the major entities involved in the system, their attributes, and the relationships established among them.

The ERD was designed to model real-world operations within a modern food delivery platform where customers place orders from restaurants, drivers deliver those orders, payments are processed digitally, loyalty systems reward customers, and predictive analytics improve delivery efficiency. The design contains multiple interconnected entities that collectively support operational processing, customer management, order tracking, delivery optimization, and analytical decision-making.

ER Diagram



Zoom the picture to get a better view of the diagram!!!

2.2 Description of Major Entities

2.2.1 CUSTOMERS Entity

- ✓ The CUSTOMERS entity stores all information related to users who place food orders through the platform.

Attributes of CUSTOMERS

Attribute	Description
customer_id	Unique identifier for each customer
first_name	Customer's first name
last_name	Customer's surname
email	Customer email address
phone	Customer contact number
loyalty_points	Accumulated reward points
tier_id	Foreign key linking customer to loyalty tier

Purpose of the Entity

- ✓ This entity serves as the central user management component of the system. It enables customer authentication, order tracking, loyalty management, and customer analytics.

Relationships

- One customer can have many addresses.
- One customer can place many orders.
- One customer can submit many ratings.
- One customer can generate many loyalty events.
- One customer may be associated with anomaly flags.

2.3.2 ADDRESSES Entity

- ✓ The ADDRESSES entity stores delivery location details for customers.

Attributes of ADDRESSES

Attribute	Description
address_id	Unique identifier for address
customer_id	References customer owner
label	Address nickname (Home, Office, etc.)
street	Street name
city	City name
sub_city	Sub-city or district
woreda	Administrative area
is_default	Indicates default delivery address

Purpose

- ✓ This entity supports multiple delivery locations for a single customer, improving convenience and delivery flexibility.

Relationship

- One customer can have many addresses.
- One address may be used in multiple orders.

2.2.3 LOYALTY_TIERS Entity

- ✓ The LOYALTY_TIERS entity manages customer membership classifications.

Attributes of LOYALTY_TIERS

Attribute	Description
tier_id	Unique loyalty tier identifier
tier_name	Tier category name
min_points	Minimum required points
discount_percent	Discount assigned to tier

Purpose

- ✓ The entity supports customer retention strategies by rewarding frequent users with discounts and benefits.

Relationship

- ✓ One loyalty tier can classify many customers.

2.2.4 RESTAURANTS Entity

- ✓ The RESTAURANTS entity stores information about restaurants registered on the platform.

Attributes of RESTAURANTS

Attribute	Description
restaurant_id	Unique restaurant identifier
name	Restaurant name
cuisine_id	References cuisine category
phone	Restaurant contact number
street	Street location
city	City name
latitude	GPS latitude
longitude	GPS longitude
is_active	Availability status
avg_prep_time_minutes	Average food preparation time

Purpose

- ✓ This entity enables restaurant registration, order preparation tracking, and menu management.

Relationships

- One cuisine type can define many restaurants.
- One restaurant can offer many menu items.
- One restaurant can prepare many orders.

2.2.5 CUISINE_TYPES Entity

- ✓ This entity categorizes restaurants according to food specialization.

Attributes of CUISINE_TYPES

Attribute	Description
cuisine_id	Unique cuisine identifier
cuisine_name	Cuisine category
description	Description of cuisine

Purpose

- ✓ Cuisine classification improves customer search experience and restaurant categorization.

Relationship

- ✓ One cuisine type can be associated with many restaurants.

2.2.6 MENU_ITEMS Entity

- ✓ The MENU_ITEMS entity stores food products offered by restaurants.

Attributes of MENU_ITEMS

Attribute	Description
item_id	Unique menu item identifier
restaurant_id	References restaurant
name	Item name
price	Item price
category	Food category
is_vegetarian	Vegetarian indicator
is_vegan	Vegan indicator
is_available	Availability status

Purpose

- ✓ This entity enables menu display, pricing management, and dietary filtering.

Relationships

- One restaurant can have many menu items.
- One menu item may appear in many order items.

2.2.7 ORDERS Entity

- ✓ The ORDERS entity represents customer purchase transactions.

Purpose

- ✓ This is the core transactional entity of the system. It coordinates customer purchases, restaurant preparation, driver delivery, payment processing, and tracking operations.

Relationships

- One customer can place many orders.
- One restaurant can prepare many orders.
- One driver can deliver many orders.
- One order can contain many order items.
- One order can have one payment.
- One order can generate assignment logs and delivery predictions.

Attributes of ORDERS

Attribute	Description
order_id	Unique order identifier
customer_id	References customer
restaurant_id	References restaurant
driver_id	References assigned driver
delivery_address_id	References delivery address
rule_id	References surge pricing rule
status	Current order status
subtotal	Cost before fees
delivery_fee	Delivery charge
surge_multiplier	Dynamic pricing multiplier
total_amount	Final payable amount
placed_at	Order placement time
estimated_delivery_time	Estimated delivery time
actual_delivery_time	Actual delivery completion time

2.2.8 ORDER_ITEMS Entity

- ✓ This entity stores detailed products contained in an order.

Attributes of ORDER_ITEMS

Attribute	Description
order_item_id	Unique identifier
order_id	References order
item_id	References menu item
quantity	Ordered quantity
unit_price	Price per item
subtotal	Calculated subtotal

Purpose

- ✓ It resolves the many-to-many relationship between orders and menu items.

Relationship

- One order contains many order items.
- One menu item may exist in many order items.

2.2.9 DRIVERS Entity

- ✓ The DRIVERS entity stores delivery personnel information.

Attributes of DRIVERS

Attribute	Description
driver_id	Unique driver identifier
first_name	Driver first name
last_name	Driver last name
phone	Driver contact number
vehicle_id	Assigned vehicle type
current_latitude	Current GPS latitude
current_longitude	Current GPS longitude
is_available	Driver availability
rating_avg	Average driver rating
total_deliveries	Total completed deliveries

Purpose

- ✓ This entity supports delivery assignment, location tracking, and performance monitoring.

Relationships

- One vehicle type can be assigned to many drivers.
- One driver can deliver many orders.
- One driver can receive many ratings.
- One driver can appear in assignment logs.

2.2.10 VEHICLE_TYPES Entity

- ✓ The VEHICLE_TYPES entity defines transportation categories.

Attributes of VEHICLE_TYPES

Attribute	Description
vehicle_id	Unique vehicle identifier
vehicle_name	Vehicle category
max_distance_km	Maximum delivery distance

Purpose

- ✓ This entity improves delivery optimization by matching vehicles to delivery requirements.

Relationship

- One vehicle type can be used by many drivers.

2.2.11 PAYMENTS Entity

- ✓ The PAYMENTS entity stores transaction and payment details.

Attributes of PAYMENTS

Attribute	Description
payment_id	Unique payment identifier
order_id	References order
payment_method	Method of payment
payment_status	Payment state
amount	Paid amount
transaction_id	Digital transaction reference
paid_at	Payment timestamp

Purpose

- ✓ This entity ensures secure financial transaction recording and payment verification.

Relationship

- One order is processed by one payment record.

2.2.12 RATINGS Entity

- ✓ The RATINGS entity stores customer feedback after order completion.

Attributes of RATINGS

Attribute	Description
rating_id	Unique rating identifier
order_id	References order
customer_id	References customer
restaurant_id	References restaurant
driver_id	References driver
food_quality_score	Food quality rating
packaging_score	Packaging rating
delivery_speed_score	Delivery speed rating
overall_score	Overall experience score

Purpose

- ✓ This entity supports service quality evaluation and customer satisfaction analysis.

Relationships

- One customer can submit many ratings.
- One driver can receive many ratings.
- One order may have one rating.

2.2.13 DELIVERY_PREDICTIONS Entity

- ✓ The DELIVERY_PREDICTIONS entity stores analytical delivery estimation information.

Attributes of DELIVERY_PREDICTIONS

Attribute	Description
prediction_id	Unique prediction identifier
order_id	References order
base_prep_time	Initial preparation estimate
distance_km	Delivery distance
distance_factor	Distance impact factor
driver_load_factor	Driver workload factor
time_of_day_factor	Traffic/time influence
predicted_minutes	Estimated delivery time
actual_minutes	Actual completion time
prediction_error	Difference between prediction and actual result

Purpose

- ✓ This entity enables predictive analytics and delivery optimization.

Relationship

- One order can have one delivery prediction record.

2.2.14 ASSIGNMENT_LOG Entity

- ✓ The ASSIGNMENT_LOG entity records driver assignment evaluation metrics.

Attributes of ASSIGNMENT_LOG

Attribute	Description
assignment_id	Unique assignment identifier
order_id	References order
driver_id	References driver
proximity_score	Distance suitability score
load_score	Driver workload score
rating_score	Driver performance score
vehicle_score	Vehicle suitability score
total_score	Final calculated assignment score
assigned_at	Assignment timestamp

Purpose

- ✓ This entity improves delivery dispatch efficiency and transparency.

Relationships

- One order can generate many assignment logs.
- One driver can appear in many assignment logs.

2.2.15 LOYALTY_EVENTS Entity

- ✓ The LOYALTY_EVENTS entity records customer reward activities.

Attributes of LOYALTY_EVENTS

Attribute	Description
event_id	Unique event identifier
customer_id	References customer
order_id	References order
event_type	Reward activity type
points_changed	Points gained or deducted
points_balance_after	Updated points balance
description	Event description

Purpose

- ✓ This entity tracks loyalty program transactions and customer reward history.

Relationship

- One customer can generate many loyalty events.

2.2.16 ANOMALY_FLAGS Entity

- ✓ The ANOMALY_FLAGS entity records suspicious or abnormal activities.

Attributes of ANOMALY_FLAGS

Attribute	Description
flag_id	Unique anomaly identifier
customer_id	References customer
order_id	References order
driver_id	References driver
flag_type	Type of anomaly
severity	Risk severity
is_resolved	Resolution status

Purpose

- ✓ This entity enhances security, fraud detection, and operational monitoring.

Relationships

- One customer may have many anomaly flags.
- One driver may have many anomaly flags.
- One order may generate many anomaly records.

2.4 Relationship Analysis

- ✓ The database contains several one-to-many relationships that ensure efficient normalization and data integrity.

Major Relationships

Relationship	Cardinality
LOYALTY_TIERS → CUSTOMERS	One-to-Many
CUSTOMERS → ADDRESSES	One-to-Many
CUSTOMERS → ORDERS	One-to-Many
RESTAURANTS → MENU_ITEMS	One-to-Many
RESTAURANTS → ORDERS	One-to-Many
DRIVERS → ORDERS	One-to-Many
ORDERS → ORDER_ITEMS	One-to-Many
MENU_ITEMS → ORDER_ITEMS	One-to-Many
ORDERS → PAYMENTS	One-to-One
ORDERS → DELIVERY_PREDICTIONS	One-to-One
CUSTOMERS → RATINGS	One-to-Many
DRIVERS → RATINGS	One-to-Many
ORDERS → ASSIGNMENT_LOG	One-to-Many

- ✓ The relationships were carefully designed to eliminate redundancy while preserving accurate data associations.

2.5 Logical Schema Design

- ✓ The logical schema translates the ER diagram into relational tables consisting of primary keys and foreign keys.

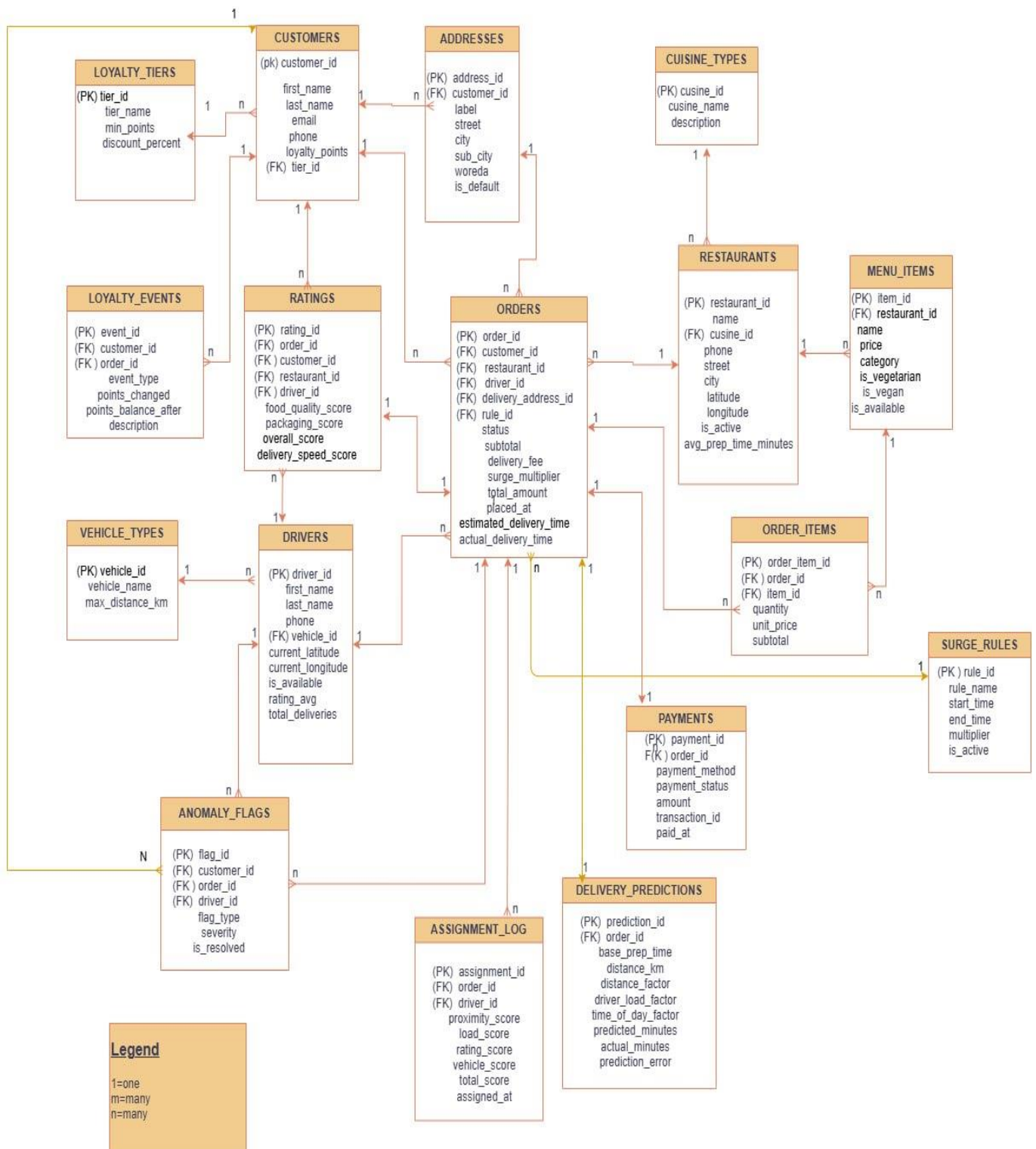
The schema ensures:

- Entity integrity through primary keys
- Referential integrity through foreign keys
- Reduced redundancy
- Efficient query execution
- Proper normalization

The logical schema includes all operational and analytical components required by the food delivery platform.

- The ORDERS table acts as the central transactional table connected to customers, restaurants, drivers, addresses, payments, and predictive modules.
- The ORDER_ITEMS table resolves the many-to-many relationship between orders and menu items.

- Analytical entities such as DELIVERY_PREDICTIONS, ASSIGNMENT_LOG, and ANOMALY_FLAGS support intelligent system functionalities including predictive analytics, automated driver assignment, and fraud monitoring.



Zoom the picture to get a better view of the diagram!!!

2.6 Normalization Process

- ✓ Normalization is a database design technique used to organize data properly by reducing redundancy and eliminating anomalies. In the development of the QuickBite Intelligent Food Delivery System database, normalization was performed carefully from the unnormalized stage up to Boyce-Codd Normal Form (BCNF).
- ✓ During the early design stage of the system, most of the information was initially imagined inside a few large tables because it looked simpler to manage. However, after analyzing the data more closely, several problems became noticeable. Customer information was repeated many times, restaurant details appeared in multiple records, and order items were stored using repeating columns. If the system had been implemented in that structure, it would have created serious consistency and maintenance problems later.
- ✓ To avoid these issues, the database was normalized step by step. At every stage, functional dependencies were analyzed, unnecessary repetition was removed, and separate relations were created where needed. The normalization process also helped improve scalability and maintain data integrity throughout the system.

The following subsections explain the complete normalization process in detail.

2.6.1 Unnormalized Form (UNF)

- Before normalization, the system data could be represented in one large table containing customer details, restaurant information, ordered items, payment information, delivery information, and rating details all together. This type of structure is common in the early stages of design before proper normalization is applied.

RAW_ORDER_DATA

Attribute	Description
order_id	Unique order identifier
customer_name	Name of customer
customer_phone	Customer phone number
customer_email	Customer email
customer_address	Delivery address
loyalty_tier	Customer loyalty tier
restaurant_name	Restaurant name
restaurant_phone	Restaurant contact
cuisine_type	Restaurant cuisine
item_1_name	First ordered item
item_1_price	Price of first item
item_1_qty	Quantity of first item
item_2_name	Second ordered item
item_2_price	Price of second item
item_2_qty	Quantity of second item
driver_name	Delivery driver
vehicle_type	Vehicle used
payment_method	Payment method
payment_amount	Amount paid
order_status	Current order status
order_date	Date of order

Problems Identified in UNF

After analyzing the table structure, several problems were identified.

1. Repeating Groups

The item columns such as:

- item_1_name
- item_2_name
- item_3_name

created repeating groups. Since an order may contain any number of items, storing them using fixed columns was not practical.

2. Data Redundancy

- Customer details were repeated for every order. For example, if a customer placed 20 different orders, the same customer phone number and email would appear 20 times.
- Restaurant details also suffered from the same problem.

3. Update Anomaly

- If a restaurant changed its phone number, every row containing that restaurant would need to be updated manually. Missing even one row would create inconsistent data.

4. Insertion Anomaly

- A new menu item could not be added unless an order already existed containing that item.
- Similarly, a new restaurant could not be inserted properly without associated order data.

5. Deletion Anomaly

- Deleting an order could accidentally remove important customer or restaurant information from the system.

For example, deleting the only order belonging to a customer could result in losing the customer's contact details as well.

- Because of these problems, the database structure required normalization.

2.6.2 First Normal Form (1NF)

Definition of 1NF

A relation is in First Normal Form when:

- Each attribute contains atomic values
- Repeating groups are removed
- Every row is uniquely identifiable using a primary key

The main goal of 1NF is to eliminate repeating attributes and ensure that each column stores only one value.

Conversion from UNF to 1NF

- ✓ To achieve 1NF, the repeating item columns were removed from the original table structure. Instead of storing ordered items in separate columns, each ordered item was stored as a separate row.
- ✓ At first, we considered keeping item details together inside the order table because it looked easier during data entry. However, after discussing how orders with many items would behave, we realized that storing items as rows would make the database more flexible and easier to query later.

Tables Created in 1NF

Table: ORDERS_1NF

✓ order_id (PK)	✓ driver_id	✓ order_status
✓ customer_id	✓ payment_method	✓ order_date
✓ restaurant_id	✓ total_amount	

Primary Key: order_id

Table: ORDER_ITEMS_1NF

order_id	item_seq	item_name	item_price	quantity
101	1	Pizza	250	2
101	2	Coke	40	1
102	1	Burger	180	1
order_id	item_seq	item_name	item_price	quantity

Primary Key: (order_id, item_seq)

Improvements Achieved in 1NF

After applying 1NF:

- Repeating item groups were removed
- Each attribute contained only atomic values
- Orders could contain any number of items
- Primary keys were properly defined

➤ Although the structure improved significantly, some dependency problems still remained.

For example: item_name and item_price did not depend on the whole composite key (order_id, item_seq).

- This resulted in partial dependency, which needed to be solved in the next normalization stage.

2.6.3 Second Normal Form (2NF)

Definition of 2NF

A relation is in Second Normal Form when:

- It is already in 1NF
- Every non-key attribute depends on the entire primary key
- No partial dependency exists

Partial dependency mainly occurs in relations that use composite primary keys.

Partial Dependency Identified

In the table:

- ORDER_ITEMS_1NF(order_id, item_seq, item_name, item_price, quantity)
- The primary key was: (order_id, item_seq)

However:

- item_name
- item_price

Depended only on the item itself and not on the full composite key. This meant that the table violated 2NF.

Functional Dependency Analysis

The following dependencies were identified:

- (order_id, item_seq) → quantity
- item_id → item_name, item_price

Since item_name and item_price depended only on item_id, they needed to be separated into another relation.

Conversion to 2NF

To eliminate the partial dependency:

- Item details were removed from ORDER_ITEMS
- A separate MENU_ITEMS table was created
- ORDER_ITEMS retained only transactional data

Tables After Applying 2NF

Table: MENU_ITEMS

item_id	item_name	item_price
1	Pizza	250
2	Coke	40
3	Burger	180
item_id	item_name	item_price

Primary Key: item_id

Table: ORDER_ITEMS

order_id	item_id	quantity	subtotal
101	1	2	500
101	2	1	40
102	3	1	180

Primary Key: (order_id, item_id)

Foreign Keys: order_id → ORDERS(order_id), item_id → MENU_ITEMS(item_id)

Additional 2NF Improvements

During dependency analysis, we also realized that customer information was unnecessarily repeated in the ORDERS table.

Attributes such as:

- customer_name
- customer_phone
- customer_email

Depended only on customer_id., because of this, a separate CUSTOMERS table was introduced.

Table: CUSTOMERS

customer_id	first_name	last_name	email	phone
-------------	------------	-----------	-------	-------

Primary Key: customer_id

This reduced redundancy and simplified future updates involving customer information.

2.6.4 Third Normal Form (3NF)

Definition of 3NF

A relation is in Third Normal Form when:

- It is already in 2NF
- No transitive dependency exists

A transitive dependency occurs when a non-key attribute depends on another non-key attribute instead of directly depending on the primary key.

Transitive Dependency Identified

One important transitive dependency appeared in the customer loyalty structure. Initially, the customer table contained:

- loyalty_tier
- discount_percent

The dependency looked like this:

➤ customer_id → tier_id → discount_percent

This meant that discount_percent depended on tier_id, not directly on customer_id.

If discount percentages changed in the future, many customer records would need to be updated.

Conversion to 3NF

To remove the transitive dependency, a separate LOYALTY_TIERS table was created.

Table: LOYALTY_TIERS

tier_id	tier_name	min_points	discount_percent
1	Bronze	0	0
2	Silver	500	5
3	Gold	1000	10

Primary Key: tier_id

Updated CUSTOMERS Table

customer_id	first_name	last_name	email	phone	loyalty_points	tier_id
-------------	------------	-----------	-------	-------	----------------	---------

Foreign Key: tier_id → LOYALTY_TIERS(tier_id)

Other Transitive Dependencies Removed

Restaurant Information

Originally, restaurant details such as:

- restaurant_name
- restaurant_phone
- cuisine_type were stored inside the ORDERS table.

These attributes depended on restaurant_id, not on order_id.

To solve this issue, a separate RESTAURANTS table was created.

Driver Information

Driver-related information including:

- driver_name
- vehicle_type also depended on driver_id.

As a result:

- DRIVERS table
- VEHICLE_TYPES table were created separately.

Address Information

- ✓ During normalization, we also discussed whether customer addresses should remain inside the ORDERS table. Finally, we decided to separate them into an ADDRESSES table because one customer may use multiple delivery addresses over time.
- ✓ This design also improved scalability for future expansion.

Final ORDERS Table After 3NF

Attribute
order_id
customer_id
restaurant_id
driver_id
delivery_address_id
subtotal
delivery_fee
total_amount
order_status
placed_at

- At this stage, every non-key attribute depended directly on the primary key.
- All identified transitive dependencies had been removed successfully.

2.6.5 Boyce-Codd Normal Form (BCNF)

Definition of BCNF

- ✓ BCNF is a stricter version of Third Normal Form.
- ✓ For every functional dependency: $X \rightarrow Y$
 - X must be a superkey.
 - In simple terms, every determinant must be a candidate key.

BCNF Verification Process

- After completing 3NF, all relations were analyzed individually to determine whether any non-superkey determinants still existed.
- Most relations already satisfied BCNF naturally because they had simple primary keys and properly separated dependencies.

Example 1: CUSTOMERS Relation

Candidate Keys:

- customer_id
- email

Functional Dependencies:

-  customer_id $\rightarrow$ first_name, last_name, email, phone, loyalty_points, tier_id
-  email $\rightarrow$ customer_id, first_name, last_name, phone, loyalty_points, tier_id

Since both determinants are candidate keys, the relation satisfies BCNF.

Example 2: ORDER_ITEMS Relation

Candidate Key:

- (order_id, item_id)

Functional Dependency:

- (order_id, item_id) $\rightarrow$ quantity, subtotal

The determinant itself is the candidate key, therefore no BCNF violation exists.

Example 3: RATINGS Relation

Candidate Key:

- rating_id

We checked whether dependencies such as: order_id $\rightarrow$ restaurant_id existed inside the relation.

After analyzing the table structure, we concluded that the dependency does not hold within RATINGS because restaurant information is stored independently. Therefore, the relation satisfies BCNF.

Example 4: ASSIGNMENT_LOG Relation

Candidate Key:

- assignment_id
- We also examined whether: order_id → driver_id could create a dependency problem.

However, one order may be reassigned to multiple drivers, meaning order_id does not uniquely determine driver_id. Because of this, no BCNF violation exists.

BCNF Verification Summary

Relation	Candidate Key(s)	BCNF Status
customers	customer_id, email	✔ Satisfied
addresses	address_id	✔ Satisfied
loyalty_tiers	tier_id	✔ Satisfied
loyalty_events	event_id	✔ Satisfied
restaurants	restaurant_id	✔ Satisfied
cuisine_types	cuisine_id	✔ Satisfied
menu_items	item_id	✔ Satisfied
orders	order_id	✔ Satisfied
order_items	(order_id, item_id)	✔ Satisfied
drivers	driver_id	✔ Satisfied
vehicle_types	vehicle_id	✔ Satisfied
payments	payment_id	✔ Satisfied
ratings	rating_id	✔ Satisfied
delivery_predictions	prediction_id	✔ Satisfied
assignment_log	assignment_id	✔ Satisfied
anomaly_flags	flag_id	✔ Satisfied
surge_rules	rule_id	✔ Satisfied

Based on the dependency analysis performed during database design, the complete QuickBite schema satisfies BCNF requirements.

2.6.6 Benefits Achieved Through Normalization

The normalization process provided several important benefits to the QuickBite system.

- ✓ **Reduced Redundancy:-** Customer, restaurant, and driver details are stored only once instead of being repeated across many orders.
- ✓ **Better Data Integrity:-** Foreign key constraints ensure that every order references valid records. This prevents inconsistent or orphaned data.
- ✓ **Easier Maintenance:-** Updates become much simpler. For example, changing a restaurant phone number requires modifying only one record.
- ✓ **Reduced Anomalies:-** Insertion, deletion, and update anomalies were minimized significantly after normalization.
- ✓ **Improved Scalability:-** The database structure became more organized and scalable for future system expansion.
- ✓ **Better Query Management:-** Smaller and more focused tables improve indexing efficiency and analytical query performance.

CHAPTER 3: IMPLEMENTATION

3.1 Database Creation and Table Structure

The QuickBite database was implemented using MySQL Workbench. All 17 tables were successfully created including core business tables, intelligence tables and reference/lookup tables. The following screenshot shows the complete table structure in the left panel of MySQL Workbench.

All 17 Tables in quickbite_db (MySQL Workbench)”



- **Result:** All 17 tables created successfully — addresses, assignment_log, anomaly_flags, cuisine_types, customers, delivery_predictions, drivers, loyalty_events, loyalty_tiers, menu_items, orders, order_items, payments, ratings, restaurants, surge_rules, and vehicle_types.

3.2 Database Views

Four views were created to simplify complex queries and provide ready-made analytical reports. Each view joins multiple tables and returns a comprehensive result set in a single query.

3.2.1. Driver Performance View (vw_driver_performance)

The following screenshot shows the result of querying the vw_driver_performance view, which combines driver data with ratings and assignment logs to produce a full driver performance report including average speed scores, behavior scores and assignment scores.

vw_driver_performance View Result:


```

1 • USE quickbite_db;
2
3 • SELECT * FROM vw_order_summary;
4 • SELECT * FROM vw_prediction_accuracy;
5 • SELECT * FROM vw_restaurant_performance;
6 • SELECT * FROM vw_driver_performance;

```

	driver_id	driver_name	vehicle_name	total_deliveries	rating_avg	orders_in_system	avg_speed_score	avg_behavior_score	avg_assignment_score
▶	1	Mikael Assefa	Motorcycle	145	4.80	2	5.00	5.00	34.35
	2	Biruk Wolde	Motorcycle	98	4.60	2	3.00	4.00	32.95
	3	Henok Tesfaye	Car	210	4.90	1	5.00	5.00	36.30
	4	Liya Mengistu	Bicycle	67	4.70	1	4.00	5.00	32.40

- **Result:** All 4 drivers returned with total deliveries, average rating, orders in system, average speed score, average behavior score and average assignment score successfully calculated.

3.3 Stored Procedures

Three stored procedures were implemented to handle the core business logic of the system: driver assignment, delivery time prediction and anomaly detection. All three procedures were called and tested successfully.

3.3.1 . Procedure 1 — sp_assign_best_driver

This procedure scores all available drivers based on four factors — proximity to restaurant, current load, rating average and vehicle type — and automatically assigns the highest scoring driver to the order. The following screenshot shows the procedure being called for order 5.

sp_assign_best_driver Result for Order 5”

```

1 • USE quickbite_db;
2
3 • -- test procedure 1: assign best driver to order 5
4 • CALL sp_assign_best_driver(5);
5
6 • -- test procedure 2: calculate prediction for order 5
7 • CALL sp_calculate_delivery_prediction(5);
8
9 • -- test procedure 3: run anomaly detection
10 • CALL sp_detect_anomalies();

```

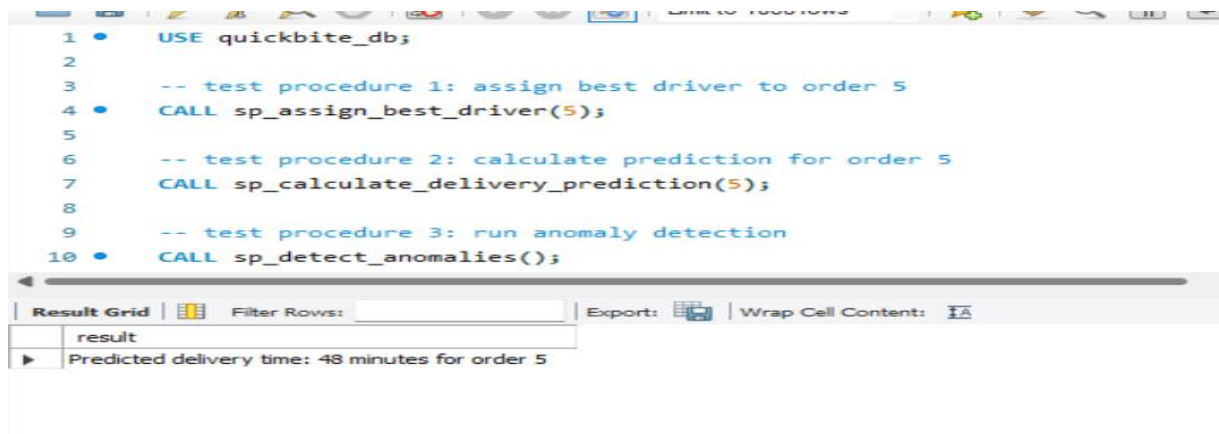
	result
▶	Driver 1 assigned to order 5 with score 36.46

- **Result:** "Driver 1 assigned to order 5 with score 36.46" — Mikael Assefa was selected as the best available driver based on proximity, load, rating and vehicle scoring.

3.3.2 Procedure 2 — sp_calculate_delivery_prediction

This procedure calculates the estimated delivery time for an order using four factors: base restaurant preparation time, distance factor, driver load factor and time-of-day factor. Each factor is stored separately in the delivery_predictions table for future accuracy analysis.

sp_calculate_delivery_prediction Result for Order 5



```
1 • USE quickbite_db;
2
3 -- test procedure 1: assign best driver to order 5
4 • CALL sp_assign_best_driver(5);
5
6 -- test procedure 2: calculate prediction for order 5
7 CALL sp_calculate_delivery_prediction(5);
8
9 -- test procedure 3: run anomaly detection
10 • CALL sp_detect_anomalies();
```

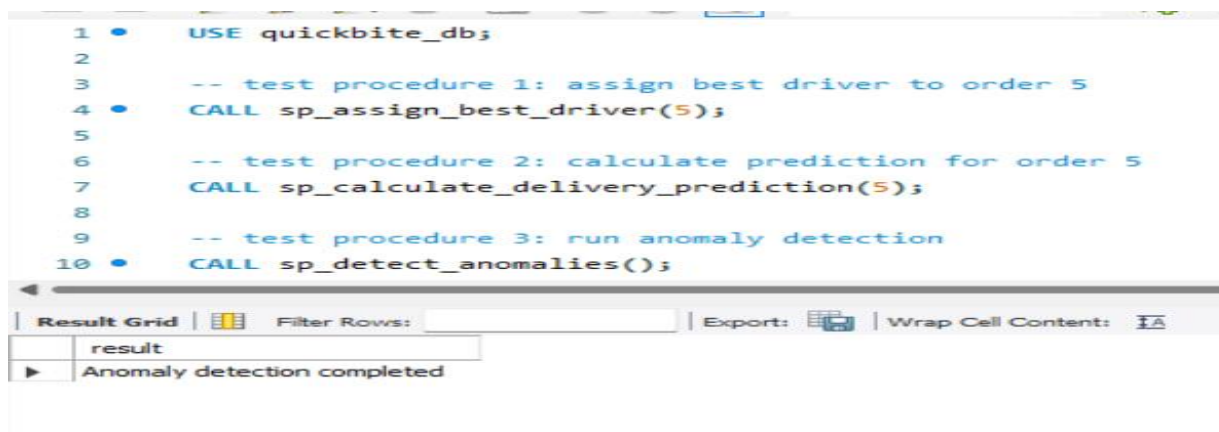
result
Predicted delivery time: 48 minutes for order 5

- **Result:** "Predicted delivery time: 48 minutes for order 5" — the prediction was calculated using all four factors and stored in the `delivery_predictions` table along with the estimated delivery timestamp.

3.3.3 Procedure 3 — `sp_detect_anomalies`

This procedure scans all orders for suspicious patterns including high refund rates, rapid duplicate orders and abnormal order values. Any detected anomalies are automatically inserted into the `anomaly_flags` table with a severity rating for management review.

`sp_detect_anomalies` Result



```
1 • USE quickbite_db;
2
3 -- test procedure 1: assign best driver to order 5
4 • CALL sp_assign_best_driver(5);
5
6 -- test procedure 2: calculate prediction for order 5
7 CALL sp_calculate_delivery_prediction(5);
8
9 -- test procedure 3: run anomaly detection
10 • CALL sp_detect_anomalies();
```

result
Anomaly detection completed

Result: "Anomaly detection completed" — the procedure successfully scanned all orders and flagged suspicious records in the `anomaly_flags` table.

3.4 Triggers

Three triggers were implemented to automate key business processes without requiring manual SQL. The trigger test below demonstrates `trg_after_order_delivered` — which automatically adds loyalty points to a customer and increments the driver delivery count the moment an order is marked as delivered.

3.4.1 Trigger Test — `trg_after_order_delivered`

The following two screenshots show customer 5 (Yonas Girma) BEFORE and AFTER order 5 was marked as delivered. The trigger fired automatically when the order status was updated.

Customer 5 Loyalty Points BEFORE Trigger (points: 900, tier: 2):

Result Grid				
Filter Rows:				
	customer_id	first_name	loyalty_points	tier_id
▶	5	Yonas	900	2
•	NULL	NULL	NULL	NULL

Customer 5 Loyalty Points AFTER Trigger (points: 929, tier: 2)

Result Grid				
Filter Rows:				
	customer_id	first_name	loyalty_points	tier_id
▶	5	Yonas	929	2
•	NULL	NULL	NULL	NULL

- **Result:** Loyalty points automatically increased from 900 to 929 (29 points = 10% of order total 290 ETB). No manual SQL was executed — the trigger handled everything automatically.

3.4.2 Loyalty Event Auto-Logged by Trigger

The trigger also automatically inserted a record into the loyalty_events table recording the points earned, the balance after, and the order reference. The following screenshot confirms this.

Loyalty Event Automatically Inserted by Trigger

Result Grid								
Filter Rows:								
	event_id	customer_id	order_id	event_type	points_changed	points_balance_after	description	created_at
▶	6	5	5	order_completed	29	929	Points earned for order #5	2026-05-02 10:52:52
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- **Result:** Event type "order_completed" automatically logged for customer 5, order 5 with 29 points earned and new balance of 929 points recorded.

3.4.3 Driver Delivery Count Auto-Updated by Trigger

The same trigger also incremented the driver's total_deliveries count automatically. Driver 1 (Mikael Assefa) had 145 deliveries before — the trigger updated it to 146.

Driver 1 Total Deliveries Auto-Incremented by Trigger

Result Grid			
Filter Rows:			
	driver_id	first_name	total_deliveries
▶	1	Mikael	146
•	NULL	NULL	NULL

- **Result:** Driver Mikael Assefa's total_deliveries automatically updated from 145 to 146 when order 5 was marked delivered no manual update required.

3.5 Advanced Analytical Queries

Seven advanced analytical queries were implemented to demonstrate the intelligence and reporting capabilities of the QuickBite system. These queries use JOINS, GROUP BY, CASE statements, window functions and subqueries.

Query 1 — Overall Delivery Prediction Accuracy Report

This query analyzes all completed delivery predictions and groups them by accuracy rating (Excellent, Good, Fair, Poor). It also calculates the overall average accuracy percentage across all orders.

Prediction Accuracy Report

	total_predictions	avg_error_minutes	excellent_count	good_count	fair_count	poor_count	avg_accuracy_percent
▶	5	9.60	2	1	2	0	81.67

- **Result:** 5 total predictions analyzed — 2 Excellent, 1 Good, 2 Fair, 0 Poor. Average accuracy: 81.67%. Average error: 9.6 minutes. This confirms the delivery prediction engine is performing well.

Query 2 — Driver Assignment Score Breakdown

This query shows the full scoring breakdown for every driver assignment decision — proximity score, load score, rating score, vehicle score and total score. This makes the assignment logic fully transparent and auditable.

Driver Assignment Score Breakdown

	assignment_id	order_id	assigned_driver	vehicle_name	proximity_score	load_score	rating_score	vehicle_score	total_score	restaurant	assigned_at
▶	7	5	Mikael Assefa	Motorcycle	NULL	NULL	NULL	NULL	36.46	Yod Abyssinia	2026-05-02 10:48:56
	3	3	Henok Tesfaye	Car	7.50	9.00	9.80	10.00	36.30	Pizza Roma	2026-05-02 10:42:38
	1	1	Mikael Assefa	Motorcycle	9.50	8.00	9.60	8.00	35.10	Yod Abyssinia	2026-05-02 10:42:38
	5	5	Mikael Assefa	Motorcycle	9.00	7.00	9.60	8.00	33.60	Yod Abyssinia	2026-05-02 10:42:38
	6	6	Biruk Wolde	Motorcycle	8.00	8.00	9.20	8.00	33.20	Pizza Roma	2026-05-02 10:42:38
	2	2	Biruk Wolde	Motorcycle	8.00	7.50	9.20	8.00	32.70	Kaldi's Coffee & Kitchen	2026-05-02 10:42:38
	4	4	Liya Mengistu	Bicycle	8.50	8.50	9.40	6.00	32.40	Tomoca Restaurant	2026-05-02 10:42:38

- **Result:** All 7 assignment records returned with full score breakdown per driver. Mikael Assefa scored highest (36.46) for order 5 and Henok Tesfaye scored highest (36.30) for order 3 due to vehicle type advantage.

Query 3 — Revenue Report with Surge Pricing Analysis

This query compares revenue generated under normal pricing versus surge pricing periods, showing the impact of the surge pricing engine on delivery fees and total revenue.

Revenue Report with Surge Analysis

Result Grid					
	pricing_type	total_orders	avg_order_value	total_revenue	avg_delivery_fee
▶	Normal Pricing	4	425.63	1702.50	53.75
	Low Surge	1	385.00	385.00	45.00
	High Surge	1	290.00	290.00	50.00

- **Result:** Normal pricing generated 1702.50 ETB across 4 orders. Low surge (1.20x) generated 385.00 ETB. High surge (1.50x) generated 290.00 ETB from 1 order showing the multiplier effect on revenue.

Query 4 — Customer Loyalty Report

This query shows every customer's current loyalty tier, points balance, how many points they need to reach the next tier, and their total spending history.

Customer Loyalty Report

Result Grid									
	customer_name	email	current_tier	current_points	next_tier	points_needed_for_upgrade	total_orders	total_spent	avg_order_value
▶	Dawit Tadesse	dawit.tadesse@email.com	Gold	1600	NULL	NULL	1	469.50	469.50
	Yonas Girma	yonas.girma@email.com	Silver	929	Gold	571	1	290.00	290.00
	Abebe Kebede	abebe.kebede@email.com	Silver	750	Gold	750	2	843.00	421.50
	Tigist Haile	tigist.haile@email.com	Bronze	200	Gold	1300	1	385.00	385.00
	Tigist Haile	tigist.haile@email.com	Bronze	200	Silver	300	1	385.00	385.00
	Selam Bekele	selam.bekele@email.com	Bronze	50	Gold	1450	1	390.00	390.00
	Selam Bekele	selam.bekele@email.com	Bronze	50	Silver	450	1	390.00	390.00

- **Result:** All 5 customers returned with tier status, current points, next tier name and points needed for upgrade. Dawit Tadesse is already Gold tier with 1600 points and no further upgrades available.

Query 5 — Restaurant Performance Ranking

This query ranks all restaurants by total revenue and average rating using SQL window functions (RANK OVER). It provides a complete performance comparison across all restaurants in the system.

Restaurant Performance Ranking

Result Grid										
	restaurant_name	cuisine_name	total_orders	avg_overall_rating	avg_food_quality	avg_packaging	total_revenue	avg_order_value	revenue_rank	rating_rank
▶	Pizza Roma	Italian	2	5.00	5.00	5.00	939.50	469.75	1	1
	Yod Abyssinia	Ethiopian	2	5.00	5.00	4.00	663.00	331.50	2	1
	Tomoca Restaurant	Ethiopian	1	4.00	4.00	3.00	390.00	390.00	3	3
	Kaldi's Coffee & Kitchen	Fast Food	1	4.00	4.00	4.00	385.00	385.00	4	3

- **Result:** Pizza Roma ranked 1st by revenue (939.50 ETB) and 1st by rating (5.00). Yod Abyssinia ranked 2nd by revenue. Both Pizza Roma and Yod Abyssinia share the top rating rank of 1.

Query 6 — Anomaly Flags Summary Report

This query summarizes all anomaly flags grouped by severity and type, showing how many are resolved vs unresolved. This serves as a management dashboard for monitoring suspicious activity.

Anomaly Flags Summary Report

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	severity	flag_type	total_flags	unresolved	resolved	affected_orders
▶	high	suspicious_location	1	1	0	NULL
	medium	abnormal_order_value	1	1	0	Order #5
	low	multiple_payment_methods	1	1	0	Order #2

- **Result:** 3 anomaly flags detected — 1 high severity (suspicious driver location), 1 medium severity (abnormal order value on Order #5), and 1 low severity (multiple payment methods on Order #2). All currently unresolved.

Query 7 — Recommendation Engine

This query implements a collaborative filtering recommendation engine in SQL. It finds items ordered by customers with similar ordering patterns that the target customer has not yet tried, ranked by popularity and average food rating.

Recommendation Engine Result for Customer 1

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	recommended_item	restaurant_name	price	category	times_ordered	avg_food_rating
	Margherita Pizza	Pizza Roma	220.00	Pizza	1	5.00

- **Result:** Margherita Pizza from Pizza Roma recommended for Customer 1 (Abebe Kebede) with a 5.00 average food rating. This item was popular with customers who ordered from the same restaurants as Customer 1 but has not been ordered by Customer 1 yet.

3.6 MONGO DB IMPLEMENTATION

MongoDB is used in QuickBite to store semi-structured, frequently changing, and log-based data where schema flexibility is beneficial. The following collections were implemented:

- driver_activity
- menu_catalog
- order_snapshots
- customer_reviews

These collections support real-time analytics, fast lookups, and hierarchical data that does not fit well into strict relational tables.

1. Driver Activity Logs Collection

In the *Quick Bite* database ecosystem, the `driver_activity_log` collection handles highly dynamic, operational data streams. While a SQL implementation relies on a normalized approach spreading data across multiple tables (such as Drivers, Shifts, and Order_Records), this MongoDB implementation leverages a **Document-Oriented, Denormalized Strategy**.

This approach consolidates an entire driver's shift lifecycle into a single BSON document. By organizing data this way, the system minimizes read latency and eliminates the computational overhead of database joins during high-frequency queries from the management dashboard.

Document Schema Design

The collection is built on a semi-structured schema capable of handling direct key-value pairs, nested sub-documents, and dynamic arrays.

Fields and Data Types:

- **_id (ObjectId):** A unique 12-byte BSON identifier automatically generated by MongoDB to serve as the document's primary key.
- **log_id / driver_id (Int32):** Indexed numerical keys used for rapid internal routing and logical mapping back to core relational assets.
- **driver_name (String):** A **denormalized** field. Storing the driver's name directly inside the log violates SQL normalization rules (2NF), but it is a critical NoSQL pattern that allows the application to render logs on a UI instantly without querying a separate Drivers collection.
- **date (String):** Temporal tracking marker formatted as YYYY-MM-DD.
- **vehicle (String):** The transportation mode utilized during the shift (e.g., "Motorcycle"), facilitating fleet-based filtering.
- **shift_start / shift_end (String) & total_hours (Int32):** Track operational availability and hours worked.

Advanced NoSQL Embedding Patterns

The true power of this MongoDB design lies in its use of hierarchical data structure embedding:

- **activity_log (Array):** Instead of writing thousands of micro-rows to an external transaction table, a sequential timeline of actions (such as order pickups, route updates, or status changes) is pushed directly into this internal array.
- **summary (Object):** An embedded sub-document containing pre-aggregated metrics calculated at the close of a shift. This ensures shift statistics are readily accessible without running expensive run-time aggregate functions like SUM() or COUNT().

```
▶ _id: ObjectId('69f5bd036a5e260a16e94153')  
log_id: 1  
driver_id: 1  
driver_name: "Mikael Assefa"  
date: "2024-01-15"  
vehicle: "Motorcycle"  
shift_start: "08:00"  
shift_end: "20:00"  
total_hours: 12  
▶ activity_log: Array (3)  
▶ summary: Object
```

```
_id: ObjectId('69f5bd036a5e260a16e94154')  
log_id: 2  
driver_id: 2  
driver_name: "Biruk Wolde"  
date: "2024-01-15"  
vehicle: "Motorcycle"  
shift_start: "09:00"  
shift_end: "21:00"  
total_hours: 12  
▶ activity_log: Array (3)  
▶ summary: Object
```

Data Analysis of Captured Instances

- **Document 1 (log_id: 1):** Captures the shift details for **Mikael Assefa**, tracking an 8:00 AM to 8:00 PM shift (shift_start: "08:00", shift_end: "20:00") totalizing exactly 12 hours.
- **Document 2 (log_id: 2):** Captures the shift details for **Biruk Wolde**, tracking a 9:00 AM to 9:00 PM shift (shift_start: "09:00", shift_end: "21:00") also totalizing exactly 12 hours.

Both records successfully contain collapsed activity_log arrays with 3 elements each and an embedded summary object, proving that the document schema is highly uniform yet scalable.

2. Menu Catalog Collection

In the *Quick Bite* database ecosystem, the core restaurant profile and food catalog are managed via the restaurants (or menu_catalog) collection. In a typical relational SQL model, managing a restaurant's profile, operational metadata, rating summaries, and specific menu items would require mapping data across multiple distinct tables such as Restaurants, Menu_Items, and Review_Aggregates linked together with foreign key constraints.

Conversely, this MongoDB implementation applies a **Document-Oriented, Single-Document Aggregation Pattern**. By storing everything related to a restaurant's identity and its food choices under a single BSON document, the system eliminates complex joins. When a user opens the *Quick Bite* app to browse a restaurant page, a single point-read fetch delivers all profile metrics, status markers, and dishes instantaneously.

Document Schema Design

The collection utilizes a dynamic, semi-structured document model featuring embedded arrays and data types chosen for optimal app performance.

Fields and Data Types:

- **_id (ObjectId):** A unique, 12-byte BSON identifier automatically generated by MongoDB to act as the primary structural key.
- **restaurant_id (Int32):** An indexed sequential identifier mapping cleanly to application routing logic.
- **restaurant_name (String):** The official brand name of the vendor (e.g., "Yod Abyssinia", "Kaldi's Coffee & Kitchen").
- **cuisine_type (String):** Classification string used to fuel the customer application's categorical search filtering (e.g., "Ethiopian", "Fast Food").
- **location (String):** The physical or zone address of the branch inside Addis Ababa.
- **is_verified (Boolean):** A flag (true/false) dictating system validation status and vendor trust badges on the user interface.
- **opening_time / closing_time (String):** Temporal operating ranges stored in HH:MM format to handle active/closed store state logic.
- **avg_prep_time_minutes (Int32):** Operational tracking value helping the system estimate pickup and delivery windows.
- **total_reviews (Int32) & avg_rating (Int32/Double):** Pre-aggregated rating metrics updated incrementally rather than computed live, preserving database resources.

Advanced NoSQL Embedding Patterns

The defining technical advantage of this MongoDB schema design is the consolidation of its nested relations:

- **menu_items (Array of Objects):** Instead of decoupling the food offerings into an isolated menu items table that requires constant query-time linking, the individual menu items are embedded directly into an array within the specific restaurant parent. This means loading a restaurant automatically loads its menu in a single database round-trip.

```
_id: ObjectId('69f5bb026a5e260a16e94140')
restaurant_id: 1
restaurant_name: "Yod Abyssinia"
cuisine_type: "Ethiopian"
location: "Bole Road, Addis Ababa"
is_verified: true
▶ menu_items: Array (3)
opening_time: "08:00"
closing_time: "22:00"
avg_prep_time_minutes: 25
total_reviews: 1
avg_rating: 5
```

```
_id: ObjectId('69f5bb026a5e260a16e94141')
restaurant_id: 2
restaurant_name: "Kaldi's Coffee & Kitchen"
cuisine_type: "Fast Food"
location: "Bole Medhaniale, Addis Ababa"
is_verified: true
▶ menu_items: Array (2)
opening_time: "07:00"
closing_time: "22:00"
avg_prep_time_minutes: 15
total_reviews: 1
avg_rating: 4
```

Data Analysis of Captured Instances

- **Document 1 (restaurant_id: 1):** Represents **Yod Abyssinia**, displaying a specialized "Ethiopian" cuisine profile on Bole Road. It holds an embedded menu_items array containing 3 distinct dishes, records an average preparation time of 25 minutes, and maintains a perfect avg_rating: 5.
- **Document 2 (restaurant_id: 2):** Represents **Kaldi's Coffee & Kitchen**, localized under a "Fast Food" cuisine category at Bole Medhaniale. It utilizes a slightly smaller embedded menu_items array containing 2 dishes, boasts a faster average preparation runtime of 15 minutes, and carries an avg_rating: 4.

3. Order Snapshots Collection

In the *Quick Bite* database ecosystem, transactional tracking is managed via the order_snapshots collection. In a typical relational SQL model, an individual transaction requires complex dependencies spanning multiple normalized tables such as Customers, Restaurants, Drivers, Order_Items, Payments, and Shipping_Logistics linked via intensive foreign key schemas.

Conversely, this MongoDB implementation introduces the **Transactional Snapshot Pattern**. Instead of running complex joins during active operations, an order's state is captured as a self-contained document. When a user or system administrator reviews an order, a single point-read fetch retrieves the entire operational, structural, and financial state of the event instantly. This prevents performance bottlenecks during peak food-ordering hours.

Document Schema Design

The collection relies on a highly integrated document model that bundles flat metadata properties alongside complex sub-documents and transaction arrays.

Fields and Data Types:

- **_id (ObjectId):** A unique, 12-byte BSON identifier automatically generated by MongoDB to act as the primary operational key.
- **order_id (Int32):** An indexed, progressive tracking key mapped straight to front-end transactional feedback logic.
- **placed_at (String):** An ISO 8601 formatted timestamp (YYYY-MM-DDTHH:MM:SSZ) providing highly precise, timezone-aware temporal tracking.
- **status (String):** State tracking strings ("delivered") that control realtime status tracking features inside client applications.

Advanced NoSQL Embedding Patterns

The architectural superiority of this collection relies entirely on its dense, multi-layer document nesting patterns:

- **customer / restaurant / driver (Objects):** Embedded sub-documents that preserve entity snapshots at the exact moment the transaction occurred. This ensures that even if a restaurant changes its name or a customer modifies their profile later, the transactional receipt history remains intact.

- **items (Array):** A nested collection containing the itemized dish details, custom options, and quantities ordered.
- **payment / pricing / delivery (Objects):** Segmented operational contexts inside the document that cleanly decouple transaction tracking (payment state), financial metrics (subtotals, tax, delivery fees), and shipping/tracking variables (delivery addresses, rider milestones).

```

▶ _id: ObjectId('69f5bc266a5e260a16e94146')
  order_id: 1
  placed_at: "2024-01-15T12:00:00Z"
  status: "delivered"
  ▶ customer: Object
  ▶ restaurant: Object
  ▶ driver: Object
  ▶ items: Array (2)
  ▶ payment: Object
  ▶ pricing: Object
  ▶ delivery: Object

_id: ObjectId('69f5bc266a5e260a16e94147')
  order_id: 2
  placed_at: "2024-01-15T13:00:00Z"
  status: "delivered"
  ▶ customer: Object
  ▶ restaurant: Object
  ▶ driver: Object
  ▶ items: Array (2)
  ▶ payment: Object
  ▶ pricing: Object
  ▶ delivery: Object

```

Data Analysis of Captured Instances

- **Document 1 (order_id: 1):** Captures an order initialized precisely at 12:00:00 UTC ("2024-01-15T12:00:00Z"). The document registers a completed lifecycle state (status: "delivered") and contains an array of 2 purchased items alongside its complete metadata object entities.
- **Document 2 (order_id: 2):** Captures a sequential order processed one hour later at 13:00:00 UTC ("2024-01-15T13:00:00Z"). It displays structural identity to the previous document, processing the same finalized lifecycle stage (status: "delivered") with an independent nested array of 2 distinct items.

4. customer_reviews

In the *Quick Bite* database ecosystem, feedback and quality assurance tracking are managed via the `customer_reviews` collection. In a typical relational SQL model, compiling a single review is resource-intensive, requiring a centralized linking table that connects foreign keys across Customers, Restaurants, Drivers, Orders, and Review_Metadata.

Conversely, this MongoDB implementation applies an **Isolated Operational Feedback Pattern**. Instead of running runtime multi-table queries to link comments to specific drivers or kitchens, each review is generated as an independent, fully contexts-aware BSON document. When a user opens a restaurant profile or a manager audits a driver's performance, the platform can display deep, itemized reviews immediately via a single point-read operation without locking or querying transactional tables.

Document Schema Design

The collection is built on a semi-structured schema tailored for textual analysis, social interaction features (votes), and multi-entity relationship mapping.

Fields and Data Types:

- **_id (ObjectId):** A unique, 12-byte BSON identifier automatically generated by MongoDB to act as the primary operational key.
- **review_id / order_id (Int32):** Indexed keys used for quick search routing and tracing feedback straight back to its original transaction root.
- **review_text (String):** Raw user-generated feedback commentary supporting flexible string content length.
- **helpful_votes (Int32):** A counter attribute used to power community validation upvoting systems on the user interface.
- **created_at (String):** An ISO 8601 formatted timestamp (YYYY-MM-DDTHH:MM:SSZ) providing accurate, timezone-aware feedback logging.

Advanced NoSQL Embedding Patterns

The architectural flexibility of this MongoDB collection relies entirely on its multi-tiered document nesting:

- **customer / restaurant / driver (Objects):** Embedded entity snapshots. Rather than forcing the frontend to issue follow-up lookups just to find out *who* wrote the review or *which* restaurant it belongs to, relevant identification attributes are securely nested inside the document.
- **scores (Object):** An embedded document containing granular rating breakdowns (e.g., food_rating, delivery_rating, driver_rating). This groups multi-criteria ratings under a single parent object rather than polluting the root document level.
- **tags (Array):** A flexible string array containing search filters or categorical tags (e.g., ["Doro Wot", "Fast Delivery", "Late Delivery"]). This allows the database to easily support dynamic text-indexing and search filtering.

```
▶ _id: ObjectId('69f5bcb06a5e260a16e9414c')
  review_id: 1
  order_id: 1
  ▶ customer: Object
  ▶ restaurant: Object
  ▶ driver: Object
  ▶ scores: Object
  review_text: "Excellent doro wot! Driver was very professional and fast."
  tags: Array (3)
  helpful_votes: 3
  created_at: "2024-01-15T13:30:00Z"
```

```
_id: ObjectId('69f5bcb06a5e260a16e9414d')
  review_id: 2
  order_id: 2
  ▶ customer: Object
  ▶ restaurant: Object
  ▶ driver: Object
  ▶ scores: Object
  review_text: "Good food but delivery was a bit late. Sandwich was still fresh though..."
  tags: Array (3)
  helpful_votes: 1
  created_at: "2024-01-15T14:30:00Z"
```

Data Analysis of Captured Instances

The image displays the active database collection status captured on January 15, 2024, demonstrating consistent schema patterns across various user experiences:

- **Document 1 (review_id: 1):** Captures positive feedback generated at 13:30:00 UTC ("2024-01-15T13:30:00Z"). The customer left an explicit review text ("*Excellent doru wot! Driver was very professional and fast.*") which received 3 helpful votes. The document contains an array of 3 parsed tags alongside its distinct metadata objects.
- **Document 2 (review_id: 2):** Captures a mixed feedback entry processed an hour later at 14:30:00 UTC ("2024-01-15T14:30:00Z"). The review text notes a logistics bottleneck ("*Good food but delivery was a bit late...*") while confirming food quality. It holds 1 helpful vote, a 3-item tags array, and maintains identical structural properties to the first record.

3.7 Implementation Summary

The QuickBite database system was successfully implemented in MySQL with all planned features working correctly. The following table summarizes what was implemented and tested:

- ✓ 17 MySQL tables created with proper constraints, primary keys and foreign keys
- ✓ 4 database views for order summary, prediction accuracy, restaurant performance and driver performance
- ✓ 3 stored procedures for driver assignment scoring, delivery prediction and anomaly detection
- ✓ 3 triggers for automatic loyalty points, driver rating updates and loyalty tier upgrades
- ✓ 7 advanced analytical queries including window functions, subqueries and CASE statements
- ✓ 4 MongoDB collections for menu catalog, order snapshots, customer reviews and driver activity logs

All source code is available in the project GitHub repository. The complete schema.sql, queries.sql, collections.json and queries.js files are organized in the /mysql and /mongodb folders respectively.